Desktop Weather Map from Open Data

Tauri · Rust · Leaflet — reading AROME weather data

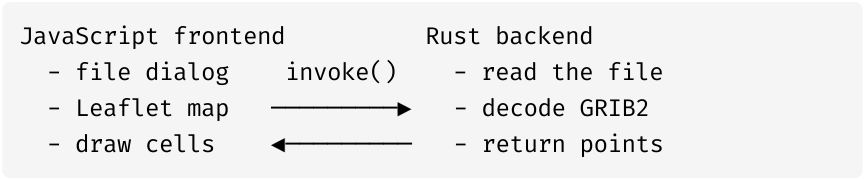
A 4-hour hands-on lab — open a real **AROME GRIB2** file, decode the **2 m air temperature** in Rust, and paint it on a map of **Montpellier**.

Use → / Space to advance · press o for the slide overview

What you will build

A **native desktop window** that:

1. Opens a `.grib2` weather file from disk
2. Decodes the 2 m temperature field in **Rust**
3. Returns `[{ lat, lon, value }]` to the UI
4. Draws one coloured cell per grid point on a **Leaflet** map



The whole app is one window: a **web UI** talking to a **Rust** program through a single `invoke()` bridge.

Why open weather data?

Open data

Data anyone can freely access, reuse and share — online, **machine-readable**, openly licensed, usually free. Weather is a flagship example: public agencies (Météo-France, NOAA, ECMWF, DWD…) publish forecasts openly.

Local uses around Montpellier — vineyard frost & irrigation · urban heat islands · sailing/wind on the Golfe du Lion · solar production.

A 2.5 km forecast, free for anyone to build on. What would *you* improve with it?

AROME — a high-resolution weather model

AROME (Météo-France)

A numerical model that divides the atmosphere into a 3-D grid and solves physics equations to forecast temperature, wind, humidity… for the coming hours.

- Horizontal grid ≈ **0.025° (~2.5 km)**
- Covers France, incl. **Montpellier** (43.61° N, 3.88° E)
- New runs at **00 / 06 / 12 / 18 UTC**
- Output shipped as **GRIB2** files
- We only need the **2 m air temperature** field
- Values are in **Kelvin** (− 273.15 → °C)

"2 m" = 2 metres above the ground — the WMO standard height for near-surface air temperature (thermometer / shelter height). It's the temperature people actually feel, and the file may store temperature at many other levels too.

What is a GRIB file?

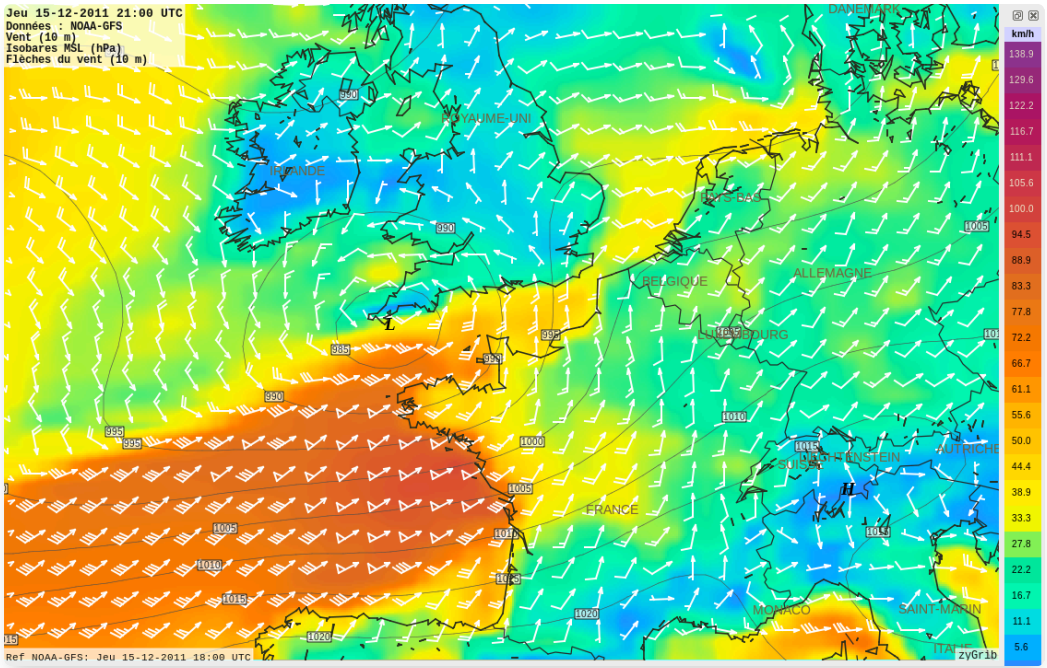
GRIB — GRIdded Binary

The WMO international standard for gridded weather data: compact, binary, and **self-describing** — each field carries the metadata needed to read it. We use **GRIB2**.

Mental model: **a stack of labelled "spreadsheets."** Each sheet is one variable, at one level, for one forecast time.

```
GRIB2 file
 ├── message ➔ one 2-D field
 │     ├── Grid Definition   : where  (corner lat/lon + step)
 │     ├── Product Definition: what   (parameter, level, time)
 │     ├── Data Representation: how   (packing method)
 │     └── Data              : the packed numbers
 └── …                                end marker "7777"
```

A GRIB file, visualised



A NOAA **GFS** file rendered in **zyGrib** — surface conditions during storm **Joachim** (15 Dec 2011).

Every coloured cell is **one grid point's value**. That is exactly what our app does with AROME's **2 m temperature** over Montpellier: decode the grid, paint a colour per cell.

Source: Wikimedia Commons — zyGrib / NOAA-GFS.

Three ideas that matter

1 — The 'what' is numbers, not text

2 m temperature = `discipline 0` / `category 0` / `parameter 0`, at fixed surface **height 2 m**. Looked up in WMO code tables.

2 — The 'where' is a grid description

The file stores corner coordinates + step. Every point's lat/lon is *derived* — the library hands it to us via `latlons()`.

3 — The data is packed (compressed)

Scaled integers, then compressed. The AROME **open-data** files we use are packed with **CCSDS/AEC** (decoded via **libaec**) — a decoder reconstructs the real numbers for us.

How the app is built

Tauri

A native window showing a **web UI** (HTML/CSS/JS) backed by a **Rust** program. Heavy work (decoding GRIB) runs in Rust; the UI runs in the webview.

The bridge

Frontend calls Rust with `invoke("name", { args })` and Rust returns JSON. That's the only "magic."

Leaflet

A slippy map of OpenStreetMap tiles. You place things by **(lat, lon)** — exactly what our decoder produces. We draw one small coloured rectangle per grid cell.

```
const points =
  await invoke("load_temperature", { path });
// → [{ lat, lon, value }, …]
```

Five parts, ~3.5 hours hands-on

Time	Part	Goal	Tag
0:30–1:00	1	Install & scaffold the app	`part1`
1:00–1:15	2	Get an AROME GRIB file	`part2`
1:15–2:15	3	Decode the temperature in Rust	`part3`
2:15–3:25	4	Show it on a Leaflet map	`part4`
3:25–4:00	5	Polish + bonus	`part5`

💡 **Each part is a git tag.** Behind, or a step won't work? Check out the tag for the part you are on and keep going — details on the next slides and at the end.

Part 1 — Install & scaffold

- Toolchains: **Rust** (rustup) + **Node.js** (LTS)
- System libs: `webkit2gtk-4.1`, `libaec`, `librsvg`, `cmake`, `clang`, `pkgconf`

```
npm create tauri-app@latest      # name it grib-map (Vanilla / JS / npm)
cd grib-map && npm install
npm run tauri dev                # first build is slow; a window opens
```

Know your files: frontend `index.html`, `src/main.js`, `src/styles.css` · backend `src-tauri/src/lib.rs`, `src-tauri/Cargo.toml`, `capabilities/default.json`.

✅ **Checkpoint 1:** the default Tauri window opens.

> 💡 **Stuck or just joined?** `git checkout part1` → a working scaffold to start from.

Part 2 — Get an AROME GRIB file

You need a `.grib2` file containing **2 m temperature**:

- **In class:** copy the file from the instructor; note its full path.
- **Download (no login):** the latest real AROME `+00H SP1` file (~16 MB) from the Météo-France open-data mirror — see `data/fetch_arome_sp1.sh`.
- **Offline:** a tiny synthetic sample is provided for testing.

✅ **Checkpoint 2:** you have a `.grib2` file and know its path.

> 💡 **No file yet?** `git checkout part2` → the `data/` helpers and a ready sample.

Part 3 — Decode the temperature in Rust

Find the field where `discipline 0 / category 0 / parameter 0`, surface `2 m`.

```rust
let latlons = submessage.latlons()?;                    // coords FIRST
let decoder = grib::Grib2SubmessageDecoder::from(submessage)?;
let values  = decoder.dispatch()?;                      // Kelvin
for ((lat, lon), kelvin) in latlons.zip(values) { /* … */ }
```

1. find the **first** 2 m temperature field
2. get coordinates **and** decode values *(order matters!)*
3. keep the **Montpellier box** `lat ∈ [43, 44.2]`, `lon ∈ [3, 4.8]`
4. convert K → °C and return `Vec<TempPoint>`

✅ **Checkpoint 3:** the project compiles and `load_temperature` exists.

💡 **Decode errors / falling behind?** `git checkout part3` → working Rust commands + tests.

Part 4 — Show it on a Leaflet map

```
const map = L.map("map", { preferCanvas: true }).setView([43.61, 3.88], 9);
const path = await open({ filters: [{ name: "GRIB2", extensions: ["grib2"] }] });
const points = await invoke("load_temperature", { path });

for (const { lat, lon, value } of points)
  L.rectangle([[lat-0.0125, lon-0.0125], [lat+0.0125, lon+0.0125]],
    { stroke: false, fillColor: colorForTemp(value), fillOpacity: 0.6 }).addTo(map);
```

Enable the dialog plugin (`npm run tauri add dialog`), colour blue (−5 °C) → red (35 °C).

✅ **Checkpoint 4:** clicking the button fills the map with coloured cells.

> 💡 **Map blank or behind?** `git checkout part4` → the full working frontend.

Part 5 — Polish & bonus

Pick what interests you:

- **Polish:** auto-fit the map (`map.fitBounds`), min/max/avg, opacity slider, colour legend
- **Forecast hour:** let the user choose a term (`prod_def().forecast_time()`)
- **Wind (harder):** decode U/V (category 2, params 2 & 3, level 10 m), draw arrows
- **Live data:** download a fresh AROME file from Rust
- **Ship it:** `npm run tauri build` → a real installer

✅ **Checkpoint 5:** a polished map with stats, legend and an opacity control.

> 💡 **Want the finished reference?** `git checkout part5` → everything, done.

The checkpoint tags — never get stuck

Every part is committed as a **git tag**. If a step won't work or you joined late, jump straight to a known-good snapshot and keep up with the class:

```
git checkout part3      # working snapshot at the end of Part 3
# … follow along, then when ready:
git checkout main       # back to the course materials
```

Tag	You get a working…
`part1`	scaffolded Tauri app
`part2`	GRIB file + `data/` helpers
`part3`	Rust decoder (`describe_grib` , `load_temperature`)
`part4`	Leaflet map frontend
`part5`	finished app + polish

Cheat-sheet — the calls you'll need

Rust — `use grib::LatLons;`

```
let r = BufReader::new(File::open(&path)?);
let g = grib::from_reader(r)?;
for (i, m) in g.iter() {
  let d   = m.indicator().discipline;
  let cat = m.prod_def().parameter_category();
  let num = m.prod_def().parameter_number();
}
```

Frontend (`src/main.js`)

```
import { open }   from
  "@tauri-apps/plugin-dialog";
import { invoke } from
  "@tauri-apps/api/core";
```

Reminders: edit `lib.rs` (not `main.rs`) · the `invoke` arg key matches the Rust parameter name · K − 273.15 → °C.

Let’s build it 🌡️🗺️

Open data → Rust → a map of Montpellier.

Stuck at any point? `git checkout part<N>` and keep going.